

Yazılım Şirketlerinde Fraktal Sahiplik

Ömer Faruk Yavuz

Ekim 2025

Giriş

En başında tek bir kişiyle başlanan, ve en sonunda tek bir kişi tarafından kontrol edilebilir yapıya yeniden yakınsayan organizasyonları nasıl tasarlayabiliriz, veyahut, hız ve sürekliliği aynı anda koruyacak, gerektiğinde tekil sahipliğe inebilen ve gerektiğinde kalıcı paylaşımı sürdürebilen organizasyonları nasıl tasarlarız?

Bu metin iki işletim modunu tanımlar:

Mod-A (Kalıcı Paylaşım) — Kurumsal ve açık kaynak bağlamında güç kalıcı olarak tabana yayılır (geometrik dizi modeli).

Mod-B (Emanet Kapasite) — Startup bağlamında darboğaz çözülünce paylar eski haline döner, primary tekrar 1 olur (operasyonel hız modeli).

Problem Tanımı

Birçok ürün veya proje, başlangıçta tek bir bireyin uçtan uca sahipliğiyle ($0 \rightarrow 1$) hayata geçirilirken, bazıları da ($1 \rightarrow N$) yapılarıyla hayata geçer. ($1 \rightarrow N$) yapıları için, Domain-Driven Design (DDD), mikroservis veya benzeri organizasyonel yaklaşımlar altında, bu sahiplik modeli büyümeyi tetikler: yeni personel alımları, yeni domain tanımları ve bu domainler etrafında yapılandırılan ekiplerin oluşması gündeme gelir.

Bu tür sorumluluk bölünmeleri kimi zaman gerekli olsa da, çoğu durumda sistem üzerinde gereksiz bir yük yaratabilir; öğrenme eğrisini doğrusal olmaktan çıkararak üstel bir hale dönüştürür ve giriş seviyesinin aşılması için gereken eşiği yükseltir. Bunun doğal sonucu olarak daha fazla personel ihtiyacı doğar, bu yeni personelin koordinasyonu için de ek kaynaklara ihtiyaç duyulur, dökümantasyonlar oluşturulur, birim testleri yazılır, performans çalışmaları yapılır, yeni bugfixler ortaya çıkar ve yapılacak işlerin sayısı üstel olarak artmaya başlar.

Ne var ki startup ve scale-up ölçeğindeki şirketlerin çoğu, bu tür genişletilmiş organizasyonel yapıları finanse edecek sermayeye sahip değildir. Buna rağmen, kurumsal yapılara benzer nitelikte bir *codebase* oluşturmak; özellikle teknik borcun erken aşamada birikmesini engellemek ve uzun vadede sürdürülebilirliği sağlamak açısından kritik önemdedir.

Çözüm

Fraktal Sahiplik modeli bu sorunlara alternatif bir yaklaşım sunar. Modelde her bir proje veya ürün $0 \rightarrow 1$ sahiplikle başlar, büyüme yalnızca ve yalnızca gerçek bir darboğaz veya performans ihtiyacı ortaya çıktığında fraktal bir bölünmeyle yönetilir. Böylece yeni personel ve ekip ihtiyacı, performans çalışmaları, birim test ihtiyacı, soyutlama vs. gibi normalde tüm projede yapılması gereken işler yalnızca gerekli alanlarda devreye girer. Oranlı sahiplik ($x\%$, $(1 - x)\%$) ve *shadow* (gölge) mekanizması, hem bireysel motivasyonu hem de sürekliliği korurken, startup ölçeğinde sürdürülebilir bir codebase'in kurumsal kaliteye yakın bir şekilde inşa edilmesini mümkün kılar. Bu yaklaşım, teknik borcun birikmesini engellerken, gereksiz personel yükünü minimize eder ve organizasyonun kontrollü, esnek, organik bir şekilde büyümesini sağlar.

Bir şirketin ya da projenin tamamında aynı “sahiplik-sorumluluk-ödül” modeli küçük birimlere (takım, ürün, modül, hatta birey seviyesine) fraktal biçimde kopyalanır. **Fractal sahiplik** bir *ekip şeması* değil; ortak DNA (platform + değerler) taşıyan, kendi kendini yenileyen bir *ekosistem/organizma* yaklaşımıdır. $0 \rightarrow 1$ aşamasındaki çekirdek ürün olgunlaşınca $1 \rightarrow N$ 'e doğru mitoz benzeri bölünmelerle yeni *mini-çekirdekler* doğar.

*

Ekip Modeli vs Ekosistem (A) Klasik Ekip Modeli	(B) Ekosistem (Fractal Ownership)
• Statik organizasyon şeması, fonksiyonel takımlar. • Sorumluluklar kutucuklarda; bağımlılıklar fazladır. • Yenilik: genellikle merkezi kararlar ve büyük projelerle. • Paylaşımlı altyapı/dil çoğu zaman zayıf bağlanır.	• Yaşayan organizma; hücreler (ürünler) otonom. • Ortak DNA: platform, tasarım sistemi, değerler. • Yenilik: küçük nükleusların $1 \rightarrow N$ çoğalmasıyla. • Paylaşımlı parçalar merkezde güçlenir; fraktallar <i>import</i> eder.

Fraktal Sahipliğin İlkeleri

Fraktal Sahiplik modeli, sistemler evrilirken hem çevikliği hem de sürekliliği güvence altına alan bir dizi ilkeye dayanır.

1. **Başlangıçta Uçtan Uca Sahiplik ($0 \rightarrow 1$).** Her ürün, proje ya da (x , $1-x$)lik kısım, başlangıçta veya sonradan, tek bir bireyin tam sorumluluğu altında geliştirilir; bu, hızı, netliği ve hesap verebilirliği artırır, koordinasyon yükünü azaltır. Bir işin tek bir kişi tarafından uçtan uca sahiplenmesini sağlar (tasarım + üretim + test...). Ancak gerçek bir problem çıktığında (ör.

iş yetiştiriyor, kalite düşüyor), bu iş ikiye bölünür. Eski sahip çoğunluğu (ör. yüzde 90) elinde tutar, yeni gelen yüzde 10 ile belli bir alt parçadan sorumlu olur. Her zaman bir “yedek” kişi vardır; asıl kişi ayrılırsa iş aksamadan devam eder. En sorunlu yüzde 10luk kısmın, yeni gelen kişiye özgürce verilebiliyor olması onu güçlü bir gölge geliştirici haline getirir.

2. **Sorun Odaklı Bölünme.** Sahiplik ancak somut bir performans darboğazı, ölçek gereksinimi veya kritik risk ortaya çıktığında bölünür; gereksiz parçalanma böylece engellenir.
3. **Oranlı Sahiplik Dağıtımı (x-y Kuralı).** Bölünme gerçekleştiğinde mevcut sahip $x\%$ oranını korur; yeni sahip, problem alanına bağlı $y\%$ oranını üstlenir ($x + y = 100\%$); bu, sürekliliği korurken sorumluluğu kontrollü biçimde genişletir.
Politika Önerisi: Çoğu bağlamda $y \leq 20\%$ pratik bir üst sınırdır; istisnalar kurul/council onayı gerektirir.
4. **Gölge Sahiplik Mekanizması.** Her sahibin atanmış bir gölgesi vardır. Birincil sahip ayrılırsa, gölge kesintisiz şekilde devralır; bu, sürekliliği korur ve sistemik riski azaltır.
5. **Teknolojik Evrim.** Yeni sahipler kendi dilimlerinde alternatif teknolojiler uygulayabilir; bu, genel bütünlüğü bozmadan organik evrimi mümkün kılar (arayüz standartlarına tabi).
6. **Birleşme (Fusion) ve Çeşitlenme (Differentiation).**

Fusion: Aynı persona/akışta çakışan iki dal B_1, B_2 , kalite kapıları sağlanınca $B^* = \text{merge}(B_1, B_2)$ ile tekleştirilir (operasyon ve kullanıcı deneyimi sadeleşir).

Differentiation: Aynı DNA’dan türeyen dal D , farklı SLO/hedef için varyanta ayrılır; arayüz sözleşmesi sabit kalır.

Legacy Geçiş ve Teknik Borç Gerekçesi

Fraktal Sahiplik modelinin başlıca motivasyonlarından biri, karmaşık sistemlerde **teknolojik yenilenme ve legacy (eski) kodun göçünü** destekleme yeteneğidir. Yazılım yapıtları çoğu zaman, bağlı oldukları framework, kütüphane ya da altyapının (örn. veritabanları, depolama sistemleri) doğal ömrünü aşar. Zaman geçtikçe şu tür problemler birikir:

- zaman-para-efor-hız nedeniyle güncellenemeyen framework’ler veya çalıştırma ortamları (runtime) desteğini yitirir,
- yetersiz test, eksik tasarım dokümantasyonu,
- net soyutlama eksikliği ve clean architecture prensiplerine uyumsuzluk,

- kestirmeler, izlenmeyen ödünler (trade-off) veya performans darboğazları biçiminde biriken teknik borç.

Geleneksel monolitik sahiplik modelleri, büyük çaplı yeniden yazımlar ya da sarsıcı geçişler olmadan bu sistemleri yeniden platforma taşımayı/modernize etmeyi zorlaştırır. Buna karşılık Fraktal Sahiplik yapısı, yerel bölünmeler yoluyla legacy bileşenlerin *kademeli ikamesini* mümkün kılar:

$$\text{Legacy Alan} \mapsto (r \cdot w, (1 - r) \cdot w)$$

burada yeni dilime, modern bir uygulama inşa etme görevi verilir (örn. eski bir framework'ten yeni bir yığına geçiş ya da bir alt sistemin yeni bir veritabanına taşınması).

Neden işe yarar?

- **Artımlı modernizasyon:** Eski ve yeni bir süre birlikte yaşar; geçiş büyük patlama (*big-bang*) yerine kademelidir.
- **Borcu sınırlama:** Her bölünme, teknik borcu sonlu bir dilime hapseder, sistemik bulaşmayı önler.
- **Süreklilik:** Gölge mekanizması, yenilenmeyi sağlarken sahiplik sürekliliğini güvenceye alır.
- **Fraktal uyum:** Her dilim gerekirse daha fazla bölünebildiğinden, legacy göçü tek seferlik dönüşüm yerine yinelenebilir, fraktallaşmış bir kalıba dönüşür.

Özetle kavram yalnızca *sahiplik ve koordinasyonu* optimize etmek için değil, aynı zamanda **legacy kod tabanlarını yeni bir evrimsel yola götürmek** için sistematik bir mekanizma sunmak üzere tasarlanmıştır. Problemler kaçınılmaz olarak zamanla büyür; Fraktal Sahiplik, yıkıcı yeniden yazımlara gerek kalmadan teknolojiyi sürekli yenilemenin kontrollü bir yolunu sağlar.

İşletim Modeli

Bu bölüm, her bireye 0→1 alanı verip kalıcılığı artıran çalışma modelini özetler.

1. **0→1 Alanı:** Her mühendise uçtan uca küçük bir ürün/özellik sorumluluğu (tasarım+inşa+devreye alma).
2. **OKR Eşlemesi:** Bireysel hedefler doğrudan şirket OKR'lerine bağlanır; her kişi işinin şirket etkisini görür.
3. **Paylaşım Ritüeli:** Haftalık demo/retro, “ne yaptım/neyi öğrendim” formatında kısa, tekrarlı paylaşımlar.
4. **Yan Proje Desteği:** Değer vaat eden side-project'ler ürünleşebilir; içeride “açık kaynak” yaklaşımı teşvik edilir.
5. **Süreklilik Güvencesi:** Her alanın *shadow*'u ve temel dokümantasyonu vardır; bilgi tek kişide hapsolmaz.

Sahiplik Sürekliliği (Shadow/Backup)

Bir kişi ayrıldığında sistemin “sahiplik 0”a düşmemesi için pratikler:

- **Primary + Shadow:** Her alanın birincil sahibi ve bir gölge/backup sahibi bulunur.
- **Hızlı Devralım:** Birincil ayrılırsa gölge otomatik devralır; atamalar güncellenir, ağırlıklar değişmez.
- **Dokümantasyon:** Yol haritası, kararlar, ödünler (trade-off) ve runbook ortak depoda güncel tutulur.
- **Rotasyon:** Yılda bir kez sınırlı süreli owner/shadow rotasyonu; *bus factor* artırılır.
- **Teşvikler:** Vesting/bonus tasarımı sürekliliği ödüllendirir (erken ayrılma maliyeti netleşir).

Matematiksel Model (Fraktal Sahiplik)

Parametreler.

- Başlangıç sahipliği (kök): $w_0 = 1$.
- Bölünme oranı: $r \in (0, 1)$ (ör. $r = 0.9$).

Bölünme Kuralı (tek adım). Ağırlığı w olan bir düğüm için,

$$\begin{aligned} w'_{\text{primary}} &= r \cdot w, & w'_{\text{new}} &= (1 - r) \cdot w, \\ w'_{\text{primary}} + w'_{\text{new}} &= w \quad (\text{Korunum}). \end{aligned}$$

k adım sonra (tek dal).

$$\begin{aligned} w_{\text{primary}}^{(k)} &= r^k, & w_{\text{slice},i} &= (1 - r) r^{i-1} \quad (i = 1, 2, \dots, k), \\ \sum_{i=1}^k (1 - r) r^{i-1} + r^k &= 1. \end{aligned}$$

$k \rightarrow \infty$ limiti.

$$\lim_{k \rightarrow \infty} r^k = 0, \quad \sum_{i=1}^{\infty} (1 - r) r^{i-1} = 1.$$

Yorum: $k \rightarrow \infty$ iken güç tamamen tabana dağılır; tek bir birey baskın kalmaz.

Startup-scaleup seviyesinde singularity istenen ve beklenendir, çünkü kar (gelir) tek bir kişiye kalır, bakım-geliştirme sorumluluğu ve ödül tek bir kişiye indirgenir. Açık kaynak veya kurumsal projelerde ise demokratikleşme beklenir, çünkü işin doğası gereği birçok kişinin içinde bulunması elzemdir.

*

Optimal r Seçimi

“Optimal r ” (yani %95–%5, %90–%10, %80–%20 gibi bölünme oranları) aslında **hedeflere göre ayarlanabilir bir tasarım parametresidir**. Bunu küçük bir matematikle şu şekilde “tasarım kuralı”na bağlayabiliriz:

*

1) En küçük işe yarar dilim ($\varepsilon_{\min}$)

Yeni gelenin aldığı pay en az şu olsun ki anlamlı bir fark yaratsın:

$$1 - r \geq \varepsilon_{\min} \quad \Rightarrow \quad r \leq 1 - \varepsilon_{\min}$$

*

2) Süreklilik/Emniyet tabanı ($R_{\min}$)

Merkezdeki (mevcut sahibin) kontrolünün altına düşmemesi gereken bir taban belirleyelim:

$$r \geq R_{\min}$$

*

3) Uygulanabilir aralık ve “optimal” seçim

Fezibil aralık:

$$R_{\min} \leq r \leq 1 - \varepsilon_{\min}$$

- Eğer bu aralık boş değilse, “optimal” r bu aralıkta seçilir.
- Eğer aralık boşsa (ör. $R_{\min} = 0.95$ ve $\varepsilon_{\min} = 0.10 \Rightarrow 1 - \varepsilon_{\min} = 0.90$), öncelik neyse o korunur (ör. emniyet öncelikliyse $r = 0.95$ seçilir).

Pratik seçim kuralı:

$$r^* = \frac{R_{\min} + (1 - \varepsilon_{\min})}{2}$$

- Hedefler emniyet/süreklilik ağırlıklıysa r aralığın sağ ucuna kaydırılır.
- Hız/dağıtım ağırlıklıysa r aralığın sol ucuna kaydırılır.

*

Örnek: Neden 95–5 / 90–10 / 80–20 doğal çıkıyor?

- Emniyet-kritik: $R_{\min} \approx 0.95, \varepsilon_{\min} \approx 0.05 \Rightarrow r^* = 0.95$
- Startup/scale-up: $R_{\min} \approx 0.90, \varepsilon_{\min} \approx 0.10 \Rightarrow r^* = 0.90$

- Açık kaynak: $R_{\min} \approx 0.80, \varepsilon_{\min} \approx 0.20 \Rightarrow r^* = 0.80$

Yani bu oranlar, tabandan (süreklilik) ve tavandan (anlamli yeni dilim) gelen iki basit kısıtın kesişimi olarak doğal biçimde ortaya çıkıyor.

*

- 4) “ T sürede demokratikleşme” hedefi için r seçimi
 T bölünmeden sonra merkez payı en fazla s olsun:

$$r^T \leq s \quad \Rightarrow \quad r \leq s^{1/T}$$

Örnekler:

- 4 bölünmede merkez $\leq 70\%$: $r \leq 0.70^{1/4} \approx 0.915$
- 4 bölünmede merkez $\leq 50\%$: $r \leq 0.50^{1/4} \approx 0.84$

*

- 5) k -derinliği ve mikro-parçalanmayı sınırlama
 İlk yeni dilim: $(1-r)$, İkinci yeni dilim: $r(1-r)$, Genel yaprak: $(1-r)r^{k-1}$.
 Mikro-parçalanma freni için:

$$(1-r)r^{k-1} \geq \varepsilon_{\text{stop}} \quad \text{değilse, daha fazla bölme yapılmaz.}$$

*

- 6) Hız-Koordinasyon değiş-tokuşunu r 'ye bağlamak

$$U(r) = \alpha r + \beta(1-r) - \gamma(1-r)^2$$

Türev 0'dan:

$$\frac{dU}{dr} = \alpha - \beta + 2\gamma(1-r) = 0 \quad \Rightarrow \quad r^* = 1 - \frac{\beta - \alpha}{2\gamma}$$

Sonuç, $[R_{\min}, 1 - \varepsilon_{\min}]$ aralığına sıkıştırılır.

Geometrik Eşleme: Sierpinski Üçgeni ile Sezgiselleştirme

Sierpinski Üçgeni, *kendi-kendine-benzerlik* ilkesinin klasik bir örneğidir: tek bir yerel kuralın (üçgeni alt üçgenlere ayırma) tekrarıyla, global ölçekte karmaşık fakat düzenli bir desen oluşur. Fraktal Ownership'te de tek bir yerel kuralı sürekli tekrarlarız:

$$w \mapsto (r w, (1-r) w),$$

yani bir düğümü iki parçaya ayırır, toplam ağırlığı korur, ve gerekirse alt parçalarda aynı işlemi tekrarlarız.

*

Eşleme (Analoji) Noktaları

- **Yerel kural** → **küresel desen**: Sierpinski şekle benzer şekilde, alt-üçgenlere ayırma; *split* ($r/(1-r)$).
- **Korunum**: Sierpinski’de alan, bizim modelde sahiplik ağırlıkları *toplama korunur*.
- **Özyineleme**: Her alt parça, aynı kuralla tekrar bölünebilir (fraktal tekrar).
- **Ölçek bağımsız sezgi**: Yapı büyüdükçe “karmaşıklık artar ama kural sabittir”; bu, $k > 1$ durumlarının bir *uyarı sinyali* olarak okunmasına da sezgisel destek verir.

Not (Birebirlik yerine analoji). Klasik Sierpinski, üçgeni *üç* alt-üçgene (ölçek $1/2$) böler; bizim ownership kuralımız *iki* parçalıdır (r ve $1-r$). Dolayısıyla Sierpinski, *tam matematiksel eşleme* değil, kavramsal bir görselleştirmedir: *yerel kuralın tekrarıyla kendi-kendine-benzer bir yapı ve dağılan “kütle/alan” sezgisi*.

*

Ölçü (Measure) Sezgisi Sierpinski üzerinde bir olasılık ölçüsü tanımlandığını düşünelim; her yinelemede “kütle” alt parçalara paylaştırılır. Bizim modelde bu paylaştırma iki parçalı ve geometriktir:

$$\underbrace{(1-r)}_{\text{ilk yeni dilim}} + \underbrace{r(1-r)}_{\text{ikinci}} + \underbrace{r^2(1-r)}_{\text{üçüncü}} + \dots = 1 - r^k \xrightarrow{k \rightarrow \infty} 1,$$

yani *kütle (ownership) tabana dağılır*, merkezi ağırlık r^k ise 0’a yaklaşır.

*

Bağlı Piramitler Zinciri (Linked-Pyramid Chain) Formülasyonu (Mod-B: Emanet Kapasite)

Tanım 1 (Fraktal Bölünme Kuralı). *Bir sahiplik düğümünün ağırlığı $w > 0$ olsun. Tek-adımlı fraktal bölünme kuralı*

$$w \mapsto (r w, (1-r) w), \quad r \in (0, 1),$$

şeklindedir. Burada r mevcut sahibin koruduğu payı, $(1-r)$ ise yeni gelen sahibin üstlendiği payı ifade eder.

Tanım 2 (Bağlı Piramitler Zinciri). *Her bir birey (çalışan/geliştirici) bir piramit düğümü ile temsil edilsin. Her piramidin tepe kısmı mevcut sahibin r payını, taban kısmı ise yeni sahibin $(1-r)$ payını gösterir. Taban payı, bir sonraki piramidin tepe payına dönüşerek doğrusal bir aktarım oluşturur; böylece yan yana dizilmiş ve birbirine bağlanmış piramitlerden oluşan bir zincir elde edilir.*

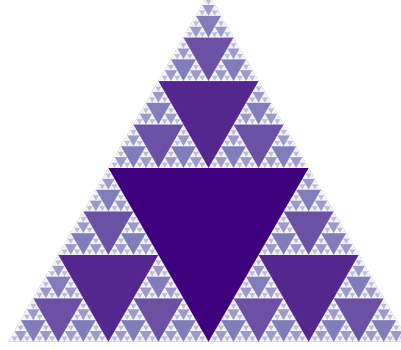


Figure 1: Klasik Sierpinski Üçgeni. Her iterasyonda kütle alt üçgenlere dağıtılır; bu durum modelimizdeki $r, (1-r)$ tabanlı geometrik dağılıma sezgisel bir analogi sunar. (Kaynak: Virtual Math Museum / Wikimedia Commons)

Bu yapı hem görsel hem de matematiksel olarak aşağıdaki şekilde ifade edilebilir:

$$\text{Piramid}_1 : (r, 1-r) \Rightarrow \text{Piramid}_2 : (r, 1-r) \Rightarrow \dots \Rightarrow \text{Piramid}_k : (r, 1-r).$$

Zincirde k adım sonra ilk (merkez) sahibin ağırlığı r^k 'ya, yeni ortaya çıkan yaprakların tipik ağırlığı ise $(1-r)r^{k-1}$ 'e orantılıdır.

Önerme 1 (Fezibil Aralık ve Seçim). *Anlamlı yeni dilim eşiği $\varepsilon_{\min} \in (0, 1)$ ve süreklilik/emniyet tabanı $R_{\min} \in (0, 1)$ verilsin.*

$$R_{\min} \leq r \leq 1 - \varepsilon_{\min}$$

aralığı boş değilse, “optimal” r bu aralık içinde hedeflere göre seçilir. Boş ise öncelikli kısıt korunur.

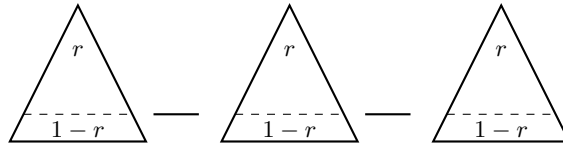


Figure 2: **Bağlı piramitler zinciri (Giza pyramids chain).** Her piramit tek bir bireyin sahiplik düğümünü temsil eder; tepe r payını, taban $(1-r)$ payını gösterir. Taban payı, bir sonraki piramidin tepesine aktarılır ve zincir bu şekilde ilerler.

*

Kapsam ve Amaç: Müdahale Odaklı Tasarım Notu

Fraktal Sahiplik, başlı başına “üstün nitelikli kod” üretme yöntemi değildir. Modelin birincil amacı, *gereken yerde, gereken zamanda, doğru müdahalenin* yapılabilmesi için sahipliği *yerel* ve *oranlı* biçimde yeniden dağıtabilen hafif bir **yönetişim ve operasyon** mekanizması sunmaktır. Kod kalitesi; mimari prensipler, arayüz standartları ve kalite kapıları (Interface Fabric, test/denetim süreçleri, gözlemlenebilirlik ve güvenlik gereklilikleri) sağlanır.

- **Öncelik:** Müdahale edilebilirlik, süreklilik ve kontrol edilebilir büyüme.
- **Araç:** $(r, 1 - r)$ oranlı bölünme ile lokal darboğazlarda hedefli kapasite/artım oluşturmak.
- **Sınır:** Fraktal bölünme tek başına “iyi kod” garantisi vermez; kalite, standart ve süreçlerle güvence altına alınır.
- **Tamamlayıcılar:** API sözleşmeleri, test/QA kapıları, gözlemlenebilirlik (log/metric/trace), güvenlik ve veri sözleşmeleri.

Özet. Bu model, “her yerde yüksek kalite üretme” iddiasından ziyade, *ihtiyaç anında doğru noktaya doğru büyüklükte* müdahale etmeyi mümkün kılan **organizasyonel bir düzenektir**; yazılım kalitesi ise bu düzenekle birlikte işletilen *standartlar ve kalite süreçlerinin* konusudur.

Arayüz Standardı – Özet

Her parça kendi teknolojisini seçebilir (örn. farklı framework, runtime, dil veya veri tabanı), ancak *bağ dokusu* sabittir. Bu bağ dokusu, tüm alt parçaların birlikte çalışmasını güvence altına alan minimum standartlar kümesidir:

$\mathcal{I} = \{\text{API sözleşmesi, kimliklendirme/erişim, gözlemlenebilirlik, data contract, dağıtım boru hattı}\}.$

Her yeni düğüm $\mathcal{I}$ 'ye (genişletilebilir) uymak zorundadır; böylece heterojen yığınlar uyumlu kalır.

Pratik Parametreler (Öneriler)

- Varsayılan oran: $r = 0.9$ (dengeli); bazı durumlarda $r = 0.8$ (hızlı dağılım) veya $r = 0.95$ (yavaş dağılım).
- Eşik örnekleri: $p95 > 500$ ms, error rate $> 1\%$, ops hours/hafta > 6 .
- Aşırı parçalanmayı önleme: $(1 - r) r^{k-1} < \varepsilon$ olduğunda yeni bölünme durdurulabilir veya komşu yapraklar birleştirilebilir.

Değişmezler (İnvariantlar)

- **Korunum.** Her bölünme yerelde korunum sağlar $\Rightarrow$ küresel toplam 1 kalır.
- **Yerel evrim, küresel düzen.** Bölünme sadece sorunlu lokasyonda olur; kontrolsüz yayılma yoktur.
- **Süreklilik.** Shadow $\rightarrow$ primary geçişiyle *bus factor* artırılır; kopma olmaz.
- **Kendine benzerlik.** Her parça aynı kurallarla yönetildiği için yapı ölçekle benzer kalır (fraktal tekrar).

*

Süreklilik Havuzu: Matematiksel Gösterim

Kurulum. Bir ana alanı A ve ilk sahibi o_0 olsun. Zaman çizelgesi üzerinde $t_0 < t_1 < t_2 < \dots$ anları tanımlayalım. Bölünme parametresi $r \in (0, 1)$ ve yeni geliştirici d_1 olmak üzere:

$$\text{(Split at } t_0) \quad w_{o_0}(t_0^+) = r, \quad w_{d_1}(t_0^+) = 1 - r, \quad w_{o_0}(t_0^+) + w_{d_1}(t_0^+) = 1. \quad (1)$$

$$\text{(Tamamlama at } t_1) \quad \text{Eğer } d_1, (1 - r) \text{ dilimini başarıyla tamamlarsa: } w_{o_0}(t_1^+) = 1, \quad w_{d_1}(t_1^+) = 0. \quad (2)$$

Burada $w_x(t)$, t anında x aktörünün A alanındaki göreceli sahiplik ağırlığını ifade eder ve yerel korunum ilkesi $w_{o_0}(t) + w_{d_1}(t) = 1$ sağlar.

İstihdam göstergesi. $E(d, t) \in \{0, 1\}$, d geliştiricisinin t anında şirkette olma durumunu gösterebilir:

$$E(d, t) = \begin{cases} 1, & \text{eğer } d \text{ şirkette ise,} \\ 0, & \text{aksi halde.} \end{cases}$$

Yeniden birikim (bottleneck) ve atama kuralı. $t_2 > t_1$ anında aynı ana alan A içinde yeni bir $(1 - r)$ 'lik darboğaz oluştuğunu varsayalım. Süreklilik havuzu kuralına göre bu yeni dilimin ataması

$$\text{assign}(A, t_2) = \begin{cases} d_1, & \text{eğer } E(d_1, t_2) = 1, \\ \text{policy_fallback}(A, t_2), & \text{eğer } E(d_1, t_2) = 0, \end{cases}$$

şeklinde olur. Yani d_1 şirkette kaldığı sürece, aynı A alanındaki tekrar eden $(1 - r)$ dilimleri otomatik olarak d_1 'e atanır.

Zincirli durum güncellemesi. Eğer $E(d_1, t_2) = 1$ ise, t_2 anında:

$$w_{o_0}(t_2^+) = r, \quad w_{d_1}(t_2^+) = 1 - r,$$

ve dilim başarıyla tamamlandığında tekrar

$$w_{o_0}(t_3^+) = 1, \quad w_{d_1}(t_3^+) = 0$$

olur. Böylece w_{o_0} değeri *olay-temelli* olarak $1 \leftrightarrow r$ arasında normalize edilip geri döner; $(1 - r)$ dilimlerinin mükerrer ataması ise d_1 üzerinde kalır.

Önerme (Süreklilik). Aynı ana alan A için $t_1 < t_2 < \dots < t_k$ anlarında doğan tüm $(1 - r)$ darboğazları verilsin. Eğer $E(d_1, t) = 1$ her $t \in \{t_2, \dots, t_k\}$ için sağlanıyorsa,

$$\forall j \in \{2, \dots, k\} : \text{assign}(A, t_j) = d_1.$$

Yani geliştirici şirketten ayrılmadığı müddetçe, aynı ana alandaki müteakip $(1 - r)$ dilimleri *daimi olarak* d_1 'e atanır.

Not. Bu durum makinesi yalnızca teknik sahiplik modelini değil, aynı zamanda *organizasyonel atama politikasını* da temsil eder. Böylece geliştirici, şirkette kaldığı sürece aynı alanın tekrar eden darboğazları için **daimi sorumlu** kabul edilir.

*

Çıkış Durumunda Sorumluluk Devri Politikası

Kapsam. Bir geliştirici d şirketten ayrıldığında ($E(d, t) = 0$), ilgili ana alan(lar)daki sorumlulukların kesintisiz devamı için aşağıdaki sıralı politika uygulanır.

*

1) Birincil Kural: Shadow Devri Her alan A için atanmış *shadow* (yedek) varsa ve uygunsa,

$$\text{assign_exit}(A, t) = \begin{cases} \text{shadow}(A), & \text{eğer shadow uygunsa,} \\ \text{policy_fallback}(A, t), & \text{aksi halde.} \end{cases}$$

Uygunluk; kapasite ($\text{load} \leq \lambda$), yetkinlik ve çıkar çatışması kontrolü ile teyit edilir.

*

2) Fallback: Aday Havuzundan Atama Shadow yoksa veya uygun değilse, aday havuzu $\mathcal{C}$ içinden seçim yapılır:

$$d^* = \arg \max_{c \in \mathcal{C}} S_A(c) \quad \text{s.t.} \quad \text{load}(c) \leq \lambda,$$

burada uygunluk skoru

$$S_A(c) = \alpha \cdot \text{domain_fit}(c, A) + \beta \cdot \text{history}(c, A) + \gamma \cdot \text{availability}(c) - \delta \cdot \text{risk}(c, A)$$

olarak tanımlanır ($\alpha, \beta, \gamma, \delta > 0$ politika katsayılarıdır).

*

3) Re-baseline ve Oranlı Yeniden Bölme Devralımın ardından alan ağırlığı yerel olarak $w = 1$ 'e normalize edilir ve ihtiyaç varsa

$$w \mapsto (r' \cdot w, (1 - r') \cdot w), \quad r' \in (0, 1),$$

ile *oranlı* bir split uygulanır. r' seçimi bağlama göre yapılır (emniyet-kritik $\Rightarrow r' \approx 0.95$; denge $\Rightarrow r' \approx 0.90$; hızlı evrim $\Rightarrow r' \approx 0.80$).

*

4) Devir Önkoşulları (Kalite Kapıları) Aşağıdaki unsurlar (genişletilebilir) tamamlanmadan devralım *tamamlanmış* sayılmaz:

- **Runbook & Dokümantasyon:** Mimari kararlar, trade-off kayıtları, test raporları, operasyon talimatları.
- **SLO/SLA Durumu:** Mevcut SLO'lar, ihlal/kaçış raporları ve açık riskler.
- **Güvenlik/Uyum:** İlgili standartlara (örn. ISO 26262/21434, IATF 16949) uyum teyidi.
- **Erişim/Yetki:** Kimlik ve erişim devri (anahtarlar, CI/CD, ortamlar) ve denetim izi güncellemesi.

*

5) Operasyonel Süreklilik (SLA Koruması) Kritik alanlarda geçici süreyle *caretaker* ataması yapılabilir:

$$\text{caretaker}(A) = \begin{cases} \text{owner}(A) & (\text{geçici}), \\ \text{platform_team} & (\text{nöbet usulü}), \end{cases}$$

ve Δt penceresi içinde kalıcı atama tamamlanır. Bu süreçte SLO'lar *degrade* etmeyecek şekilde izlenir.

Not. Bu durum makinesi yalnızca teknik sahiplik modelini değil, aynı zamanda *organizasyonel atama politikasını* da temsil eder. Böylece geliştirici, şirkette kaldığı sürece aynı alanın tekrar eden darboğazları için **daimi sorumlu** kabul edilir; ayrılık halinde ise yukarıdaki *shadow* $\rightarrow$ *fallback* düzeniyle kesintisiz süreklilik sağlanır.

Gösterimsel Figürler

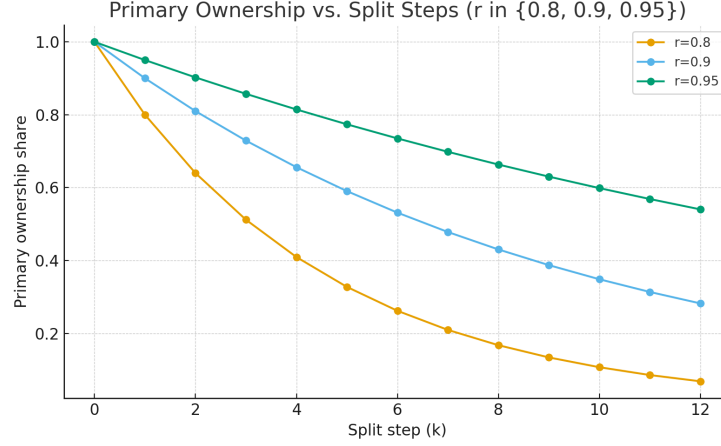


Figure 3: Farklı oranlar ($r = 0.8, 0.9, 0.95$) için split adımı k boyunca birincil sahiplikteki azalma. r arttıkça sahiplik yoğunluğu daha uzun süre korunur; r azaldıkça demokratikleşme hızlanır.

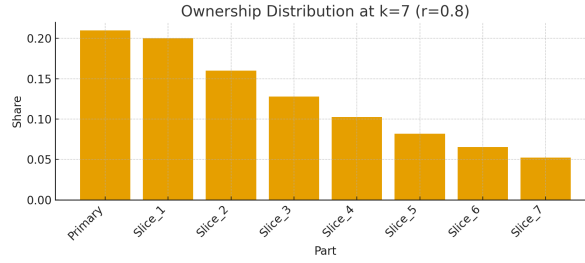


Figure 4: $r = 0.8$ için $k = 7$ sonrasında sahiplik dağılımı. Güç hızlı yayılır; birincil sahibin payı hızla azalır ve yeni dilimler anlamlı ağırlıklar kazanır. (Açık kaynak projeler için uygun)

Oran r	Bağlam	Etki / Notlar
0.95	Kurumsal / yüksek riskli alanlar	Güç yavaş dağılır; kontrol merkezde daha uzun kalır; hızlı demokratikleşme istenmiyorsa uygundur.
0.90	Varsayılan (startup/scale-up)	Denge noktası; süreklilik korunur, yeni dilimler anlamlı pay alır. Önerilen referans.
0.80	Topluluk / açık kaynak	Güç hızlı dağılır; katılım ve yayılım hızlanır; tutarlılık/disiplin gereksinimi artar.

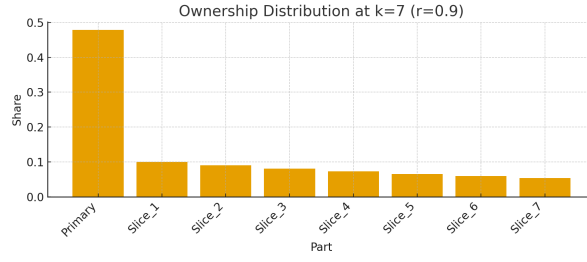


Figure 5: $r = 0.9$ için $k = 7$ sonrasında sahiplik dağılımı. Dengeli senaryo: birincil sahip çoğunluğu korur, yeni dilimler kademeli birikir. (Startup/scale-up ortamları için uygun)

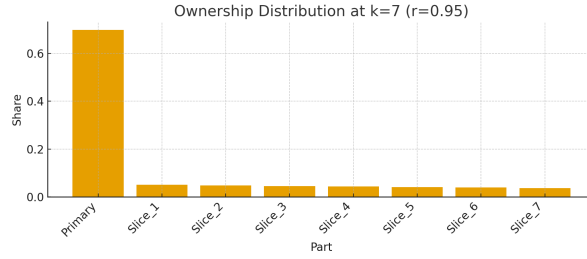


Figure 6: $r = 0.95$ için $k = 7$ sonrasında sahiplik dağılımı. Kontrol yoğun kalır; yeni dilimler çok küçük paylar alır. (Yüksek riskli veya kurumsal bağlamlar için uygun)

Gizli Backlog Problemi: 1 İş $\neq$ 1 İş Yüğü

Açıklama

Her geliştirici tarafından tamamlanan iş (bug fix veya feature) sadece görünen kısmı ifade eder. Arka planda zincirleme yeni işler doğar: test, dokümantasyon, refactor, observability, security check, review, ops vb. Dolayısıyla bir iş $\Rightarrow$ ek işlerin tetikleyicisi olur. Bu sayı sabit değil, dağılımsaldır.

Matematiksel Model

Her iş için toplam iş yükünü:

$$\text{RealCost}(1) = 1 + X,$$

olarak tanımlayalım. Burada:

- 1: Görünür iş (feature/bug),
- X : Gizli ek işlerin toplamı (rastgele değişken).

Alt Bileşenler. X aşağıdaki ek görevlerin toplamıdır:

$$X = T_{\text{test}} + T_{\text{doc}} + T_{\text{refactor}} + T_{\text{obs}} + T_{\text{review}} + T_{\text{ops}} + T_{\text{security}}.$$

Her bileşen için Bernoulli/Binomial dağılımı kullanılabilir:

$$T_j \sim \text{Bernoulli}(p_j) \cdot c_j$$

- p_j : İlgili ek işin ortaya çıkma olasılığı,
- c_j : Çıktığında getirdiği iş yükü (sabit/ortalama değer).

Beklenen Değer. Toplam beklenen ek iş yükü:

$$\mathbb{E}[X] = \sum_j p_j \cdot c_j.$$

Dolayısıyla:

$$\mathbb{E}[\text{RealCost}(1)] = 1 + \sum_j p_j \cdot c_j.$$

Tipik Senaryo (Örnek).

$$\begin{aligned} p_{\text{test}} &= 0.9, & c_{\text{test}} &= 2, \\ p_{\text{doc}} &= 0.7, & c_{\text{doc}} &= 1, \\ p_{\text{refactor}} &= 0.5, & c_{\text{refactor}} &= 2, \\ p_{\text{obs}} &= 0.6, & c_{\text{obs}} &= 1, \\ p_{\text{review}} &= 1.0, & c_{\text{review}} &= 1, \\ p_{\text{ops}} &= 0.4, & c_{\text{ops}} &= 1, \\ p_{\text{security}} &= 0.3, & c_{\text{security}} &= 2. \end{aligned}$$

Beklenen ek yük:

$$\mathbb{E}[X] = (0.9 \cdot 2) + (0.7 \cdot 1) + (0.5 \cdot 2) + (0.6 \cdot 1) + (1.0 \cdot 1) + (0.4 \cdot 1) + (0.3 \cdot 2).$$

$$\mathbb{E}[X] = 1.8 + 0.7 + 1.0 + 0.6 + 1.0 + 0.4 + 0.6 = 7.1.$$

Dolayısıyla:

$$\mathbb{E}[\text{RealCost}(1)] \approx 8.1$$

Yani “1 iş” gerçekte ortalama $\approx 8x$ iş yükü doğurur (bu oran pek tabii genişletilebilir). x , burada diğer alt problemler için de sadece soyut bir değer olarak kullanılmıştır.

Grafiksel Gösterim

Aşağıdaki grafikte X eksenı görünen iş sayısını, Y eksenı ise *beklenen* gerçek iş yükünü göstermektedir. Mavi doğru naif yaklaşımı ($y = x$), kırmızı doğru ise gerçeğı ($y = 8.1x$) temsil eder.

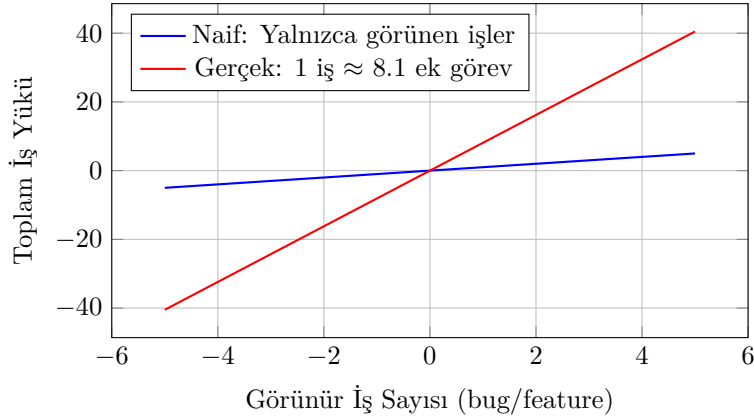


Figure 7: Her işin ardında beklenen gizli backlog: 1 iş ≈ 8 ek görev.

Interface Fabric – Ayrıntılar (Çok Teknoloji Uyumu)

Farklı teknolojilerle yerel evrimi desteklerken bütünlüğü koruyan *Interface Fabric* şunlardan oluşur. Her yeni düğüm bu kurallara **zorunlu** olarak uyar.

API Sözleşmesi

- **Protokol:** REST (JSON) veya gRPC (proto). Tek servis içinde karışık kullanım yok.

- **Versiyonlama:** /v1, /v2 gibi *URI tabanlı*; majör değişiklikler yeni sürüm olarak yayınlanır.
- **Sözleşme Kaynağı:** OpenAPI/Proto dosyası tek gerçek kaynaktır; CI'da *contract test* zorunlu.
- **İdempotensi:** Idempotency-Key desteği (özellikle ödeme/yan etkili işlemler için).
- **Hata Modeli:** Sabit hata zarfı: { code, message, correlation_id, details[] }; semantik kod tablosu paylaşımlı.
- **Paginasyon/Filtreleme:** limit/offset veya cursor-tabanlı; sıralama için sort= alanları standart.
- **Oran Sınırı:** Yanıt başlıklarında X-RateLimit-* ve Retry-After; ihlal durumunda deterministik 429.

Kimlik ve Yetki (AuthN/AuthZ)

- **Taşıma:** mTLS veya TLS1.2+ & HSTS.
- **Kimliklendirme:** OIDC/OAuth2 (JWT, kısa ömür); servisler arası *m2m* için client credentials.
- **Yetkilendirme:** *Least privilege*; rol/claim tabanlı kontroller; denetim izi (*audit log*) zorunlu.
- **Sırlar:** KMS/secret manager; hiçbir sır repo/ENV dosyasında düz metin değil.

Gözlemlenebilirlik

- **Tracing:** OpenTelemetry; trace_id, span_id, correlation_id zincirlenir.
- **Metrics:** latency_ms, error_rate, rps, saturation (CPU/mem/queue); Prometheus uyumlu.
- **Loglama:** Yapısal JSON log; zorunlu alanlar: timestamp, level, service, trace_id, user_id, error_code.
- **SLO/SLA:** Her servis için SLI'ler ve SLO hedefleri tanımlı; ihlalde uyarı kuralları (p95/p99 gecikme, hata oranı).

Performans & Dayanıklılık

- **Zaman Aşımı/Deadline:** İstemci tarafı *deadline propagation*; her çağrıda makul timeout.
- **Retry & Jitter:** İdempotent isteklerde üstel geri çekilme + jitter; non-idempotent isteklerde *ops* onayı.

- **Circuit Breaker & Bulkhead:** Her istemci kütüphanesinde aktif; geri dönüş (fallback) stratejileri belgeli.
- **Sırt Basıncı (Backpressure):** Kuyruk/işleyici sınırları; aşımında 429/503 ile hızlı başarısızlık.

Veri ve Olaylar

- **Şema Evrimi:** *Backward-compatible* değişiklik ilk tercih; kırıcı değişiklikler yeni şema sürümünde.
- **Veri Sözleşmeleri:** Tip güvenliği, *nullable* kuralları, *enum* sabitleri; *data quality* metrikleri (eksik/çakışma).
- **Olaylaşma:** Event şemaları (Avro/Proto/JSON-Schema) versiyonlu; *at-least-once* tüketim için idempotent işleme.
- **Zaman & Bölge:** Tüm timestamp'ler ISO-8601 UTC; yerelleştirme sadece sunum katmanında.

Uyumluluk & Versiyonlama

- **Deprecation Politikası:** Yayından kaldırma duyurusu, göç rehberi ve en az N hafta çift sürüm desteği.
- **Uyum (Compliance):** GDPR/KVKK veri saklama, maskeleyme/anonimleştirme; *PII tagging* ve *data lineage*.

Test & Dağıtım Boru Hattı

- **Kalite Kapıları:** Lint, SAST/DAST, lisans taraması, birim (%80+), entegrasyon ve *contract* testleri zorunlu.
- **Dağıtım:** Canary/blue-green; *health probe*'lar; otomatik geri alma (auto-rollback) kriterleri dokümanlı.
- **Kaos & Yük Testi:** Kritik servisler için periyodik *chaos* ve yük testleri; raporlar paylaşımlı.

Uygulama Notu. Farklı diller/çatıların kullanımı serbesttir; ancak yukarıdaki *Interface Fabric* maddeleri **zorunlu** kabul kriterleridir. Aykırılıklar için mimari kurul onayı ve *sunset* planı gerekir.

Demokratiklik Düzeyi: Bağlama Göre Sahiplik (Policy)

*

İlke “Demokratiklik” bağlama bağlı bir tasarım değişkenidir. *Startup/scale-up* için hız ve tekil hesap verebilirlik önceliklidir; *kurumsal* ve *açık kaynak* ekosistemlerde ise geniş katılım ve konsensüs daha sürdürülebilir olabilir.

Table 1: Bağlama göre karar tarzı ve önerilen sahiplik yapısı

Bağlam	Karar Tarzı	Önerilen r
Startup/Scale-up	Tek DRI + Shadow, DAC ¹	0.85–0.90
Kurumsal	DRI + <i>review board</i> / komite	0.95
Açık Kaynak	<i>Maintainer council</i> + RFC/konsensüs	0.80

*

Neden Startup’ta Demokratik Olmamalı?

- **Hız bariyeri:** Konsensüs arayışı karar süresini uzatır, fırsat maliyetini artırır.
- **Dağınık hesap verebilirlik:** Çok taraflı kararlar sorumluluğu bulanıklaştırır.
- **Koordinasyon yükü:** Küçük ekiplerde tartışma/seremoni maliyeti çıktıyı gölgeler.
- **Yanıl ve düzelt ilkesi:** Erken aşamada hızlı iterasyon, doğruyu “tartışarak” değil “deneyerek” bulur.

Startup Kuralı. Tek DRI + Shadow, “danış, karar ver, açıkla” (DAC), ardından *disagree-and-commit* kültürü. Sahiplik derinliği çoğunlukla $k \in \{0, 1\}$ tutulur; $k > 1$ her artışı *alarm* sayılır.

Kurumsal/Açık Kaynak Kuralı. Geniş katılım ve standart kurullar kabul edilebilir; bunun karşılığında *Interface Fabric* disiplininin ve gözlemlenebilirlik metriklerinin (*SLO*, *change review*) daha sıkı işletilmesi gerekir.

¹DAC: *Danış, Karar Ver, Açıkla* (*Consult, Decide, Communicate*).

Yönetişim: Arayüz & Standartlar

Farklı teknolojilerle yerel evrimi desteklerken bütünlüğü koruyan en az kurallar:

- **API Sözleşmesi:** REST/gRPC şemaları, versiyonlama politikası ve geriye uyumluluk ilkeleri.
- **Gözlemlenebilirlik (Observability):** Log/trace/metric zorunlu alanları (örn. request-id, user-id, latency, error-code).
- **Güvenlik:** Kimlik/erişim (*authz/authn*), gizli anahtar yönetimi, *en az ayrıcalık (least privilege)*.
- **Değişim Yönetimi:** Geriye uyumsuz değişiklikler için *deprecation* penceresi ve geçiş planı.
- **Test Standartları:** Tüm servisler için minimum test kapsamı (%80+ birim testi, kritik yol için entegrasyon/e2e). API sözleşme testleri zorunludur.
- **Dokümantasyon:** Her modül için hafif ama güncel dokümantasyon: API şemaları, önemli ödün kararları ve runbook'lar.
- **Dağıtım Hatları:** CI/CD süreçlerinde ortak kalite geçitleri (lint, güvenlik taraması, otomatik testler) ve yayına alma stratejisi (canary/blue-green).
- **Veri Sözleşmeleri:** Ortak veri şemaları, tip güvenliği, geriye/ileri uyumluluk kuralları. Veri kalitesi (null, çoğaltma, kısıt) metrikleri izlenir.
- **Dayanıklılık Kalıpları:** Tüm servisler için minimum hata toleransı (time-out, retry, circuit breaker) tanımlı olmalı.
- **Performans Bütçeleri:** API gecikmesi, throughput, hata oranı için eşikler önceden belirlenir ve SLA/SLO'larla ilişkilendirilir.
- **Birlikte Çalışabilirlik:** Farklı dil/teknolojide servisler bile ortak iletişim protokolü ve kimlik mekanizmasıyla bağlanmak zorundadır.
- **Uyumluluk (Compliance):** GDPR, HIPAA, KVKK vb. için minimum loglama, veri saklama ve anonimleştirme standartları.
- **Onboarding Oyun Kitapları:** Yeni owner/shadow devralımında izlenecek hazır kontrol listeleri ve adım adım süreçler.

Birçok Paralel $0 \rightarrow 1$ Birimiyle $1 \rightarrow N$ Ölçekleme

Fikir. Tek bir büyük $1 \rightarrow N$ geçişi yerine, hedef sistemi *birçok küçük ve özerm $0 \rightarrow 1$ birimine* bölerek ölçeklemek. Her birim tek bir sahip tarafından uçtan uca inşa edilir; *arayüz dokusu* (API/observability/auth) sayesinde birimler birleşebilir.

Operasyonel Kalıp

1. **Ayrıştırma:** Üst hedefi M bağımsız (veya zayıf bağımlı) alt yeteneğe ayır:

$$\text{Ürün} \approx \bigoplus_{j=1}^M \text{Yetenek}_j$$

2. **0→1 Sahipleri Ata:** Her Yetenek_j için tek bir $0 \rightarrow 1$ *sahibi* atanır (gölge ile).
3. **Arayüz Dokusu:** Tüm birimler ortak API sözleşmesi, gözlemlenebilirlik ve güvenlik standartlarına uyar.
4. **Kısa Kadans:** 1–2 haftalık entegrasyon ritmi (sözleşme testleri, canary/blue–green).
5. **İnce Koordinasyon:** PM/TL yalnızca arayüzleri ve bağımlılıkları koordine eder; takımlar özerktir.

Basit Çıkış (Throughput) Modeli

Her birim kendi temposunda $0 \rightarrow 1$ üretir. Bir *release train* yaklaşımıyla birleşik teslimat süresi yaklaşık:

$$T_{\text{toplam}} \approx \max_{j \in \{1, \dots, M\}} T_{0 \rightarrow 1}^{(j)} + \Delta_{\text{entegrasyon}}$$

Burada $\Delta_{\text{entegrasyon}}$ entegrasyon ve sertifikasyon tamponudur (sözleşme/e2e testleri, güvenlik taramaları).

Kalite Geçitleri (Asgari Ortak Kurallar)

- **Sözleşme Testleri:** Her birim, dışa açık API'leri için *consumer-driven contract* testleri sağlar.
- **Gözlemlenebilirlik SLO'ları:** Her birim için gecikme, hata oranı, doyum metrikleri yayımlanır (SLO & alarm).
- **Güvenlik Tabanı:** AuthN/AuthZ, gizli yönetimi, bağımlılık taraması zorunludur.
- **Veri Sözleşmeleri:** Şema versiyonlama ve geriye uyum kuralları; veri kalitesi kontrolleri.

Yönetişim Sezgileri

- **Küçük, net kapsam:** Her $0 \rightarrow 1$ birimi tek bir akışın sonuna kadar sorumludur (scope creep'i engeller).
- $k = 0/1$ **hedefi:** Birim içinde $k > 1$ görülürse *inceleme* tetiklenir (tasarım/borç/kapsam).

- **Rotasyon ve Gölge:** Süreklilik için owner→shadow rotasyonu; bus factor artırılır.
- **Teknoloji Esnekliği, Arayüz Disiplini:** Yığın özgürlüğü *Arayüz Dokusuna* uyum şartına bağlıdır.

Neden İşe Yarar

- **Paralellik:** Birçok 0→1 aynı anda ilerler ⇒ teslim süresi *en yavaş birim + entegrasyon* ile sınırlanır.
- **Sahiplik ⇒ Hız:** Tek kişi uçtan uca sahiplik, hız ve hesap verebilirlik sağlar.
- **Fraktal Tutarlılık:** Her birim, gerekirse kendi içinde de fraktal bölünebilir; model her ölçekte aynıdır.

Kural. 1→N hedefi için önce *çok sayıda küçük 0→1* yarat; birleşimi *Arayüz Dokusu* ve *release train* ile yap. Birim içi k derinliğini çoğunlukla 0 veya 1’de tut; $k > 1$ her artışı *alarm* olarak yorumla.

Sahiplik Derinliği Metriği (k)

Fraktal Sahiplik modelinde sahiplik derinliği k , organizasyonel sağlığın yapısal bir göstergesidir. k , özgün (ilk) sahibinden itibaren yapılan özyinelemeli bölünmelerin sayısı olarak tanımlanır.

- $k = 0$: İdeal başlangıç noktası. Tek kişi
- $k = 1$: Sağlıklı büyüme. Bir darboğaz veya risk bir bölünmeyi tetikler (örn. 90–10). Bu, sistemin ölçeklenmesinin doğal bir işaretidir.
- $k > 1$: Olası uyarı sinyali. Aynı dalda tekrarlanan bölünmeler altta yatan sorunlara işaret eder:
 - Aşırı teknik borç veya zayıf tasarım kalitesi.
 - Zayıf/eksik arayüz standartları.
 - Aşırı yüklenmiş veya etkisiz sahiplik.
 - Organizasyonel kapsam uyumsuzluğu.

Yönetişim Kuralı. Herhangi bir alt ağaçta $k > 1$ gözlenirse bir *inceleme* tetiklenmelidir:

1. Teknik postmortem (mimari, tasarım, borç) yapılır.
2. Organizasyonel inceleme (kapsam, iş yükü dağıtımı) yapılır.
3. Refaktör, dilimleri birleştirme veya standartları güçlendirme kararı alınır.

Yorum. Böylece sahiplik derinliği yalnızca yapısal bir sonuç değil, aynı zamanda bir *gözlemlenebilirlik metriği*dir. Sürdürülebilir organizasyonlar $k = 0$ veya $k = 1$ bölgesinde kalmayı hedefler; $k > 1$ müdahale gerektiren sistemik stresi vurgular.

Riskler ve Karşı Önlemler

- **Aşırı Parçalanma:** Dilim payı $(1 - r)r^{k-1} < \varepsilon$ ise yeni bölünme durdurulur veya yakın yapılar birleştirilir.
- **Teknoloji Kaosu:** Çekirdek standartlara uyum zorunludur; tasarım/PR incelemeleriyle rehberlik edilir.
- **Adalet Algısı:** Oran değişiklikleri ve bölünme tetikleyicileri şeffaf kurallarla önceden ilan edilir.

İnsan ve Yaşam Merkezli Tasarım İlkesi

Bu model, *insanı ve yaşamı merkeze alan* bir yaklaşımın parçasıdır. Amaç, yalnızca teknik ölçeklenebilirlik değil; geliştiricinin deneyimi, ekiplerin sürdürülebilir hızı ve kullanıcının uçtan uca memnuniyetidir. Bu nedenle Fraktal Ownership:

- **Deneyim Odaklıdır:** Ürün, ekip ve geliştirici deneyimi (DX) birlikte optimize edilir; süreçler insan yükünü azaltacak şekilde tasarlanır (daha az koordinasyon, daha net sorumluluk, daha yüksek otonomi).
- **Sağlıklı Tempo ve Sürdürülebilirlik:** Kısa vadeli hız ile uzun vadeli bakım yükü dengelenir; split yalnızca *gerçek* darboğazlarda yapılır.
- **Şeffaflık ve Adalet:** Sahiplik oranları, tetikleyiciler ve rotasyon kuralları önceden ilan edilir; güven kültürü korunur.
- **Öğrenme ve Gelişim:** Shadow/rotasyon mekanizmaları bireysel gelişimi teşvik eder; bilgi kurumsal belleğe akar.

*

Experience Architect Rolü

Experience Architect (EA), organizasyonun “deneyim mimarı”dır:

- Fraktal bölünmelerin *insan ve kullanıcı deneyimi* üzerindeki etkisini gözetir,
- *Interface Fabric* (API, observability, güvenlik) standartlarının kullanımıyla ekipler arası sürtünmeyi azaltır,
- DX/SLO/anket gibi *deneyim metriklerini* izler ve iyileştirme döngülerini yönetir,

- $1 \rightarrow N$ dönüşümünü “çoklu $0 \rightarrow 1$ birimleri” ile insan-merkezli biçimde koordine eder.

Bu perspektif, Fraktal Ownership’in yalnızca bir *teknik mimari* değil, aynı zamanda *insan ve hayat merkezli* bir *organizasyon tasarımı* olduğunu vurgular.

Kişilerin Darboğaz Çözme Kapasitesi

Fraktal Sahiplik modelinde, darboğazlar (%10'luk kısımlar) kişilerin hızlı ve kaliteli çözüm üretmesi, organizasyonun sürdürülebilirliği açısından kritik önemdedir. Ancak yeni giren, tam sisteme hâkim olmadan dar bir alanı sahiplenmek zorunda kalır. Bu durum doğal olarak riskler barındırır da, doğru tasarlanmış bir süreçle fırsata dönüştürülebilir.

Problemi Doğru Çerçeveleme

Yeni giren kişilerin eğitilmesi gereken ilk nokta, problemi küçültme ve dar bir alana odaklanma becerisidir. Bunun için şu dört adımlı şablon önerilir:

1. Gözlenen durum (mevcut gerçeklik),
2. Beklenen durum (ideal hedef),
3. Gap/Fark (aradaki boşluk),
4. Etki (kime/neyi etkilediği).

Bu şablon, sorunun tüm sistemi bilmeden izole edilmesini sağlar.

Domain’in Dinamik Olarak Belirlenmesi

Klasik Domain-Driven Design (DDD) yaklaşımında domain’ler genellikle önceden tanımlanırken, fraktal sahiplik yaklaşımında ise domain, önceden belirlenen bir çizim değil, darboğazın kendisi tarafından şekillenir:

$$\text{Domain} = \text{Bottleneck}$$

Yani sistemin hangi noktasında darboğaz ortaya çıkıyorsa, o alan yeni bir domain olarak tanımlanır. Bu sayede domain sınırları, organizasyonun gerçek problemleri tarafından dinamik biçimde belirlenir. Domain haritası statik değil, organik olarak evrilen bir yapıya dönüşür.

Sonuç

Bu metin, yazılım organizasyonlarında hız ve sürekliliği aynı anda korumaya yönelik **Fraktal Sahiplik** modelini ortaya koymaktadır. Modelin temel fikri, tek ve basit bir yerel kuralın ($w \mapsto (r w, (1 - r) w)$) tekrarıyla, sistem genelinde

hem **çeviklik** hem de **süreklilik** üreten, kendi-kendine-benzer bir yapı kurmaktır. Başlangıçta **0→1 uçtan uca sahiplik** netlik ve hız sağlar; yalnızca *gerçek* darboğazlarda **oranlı bölünme** yapılır. **Shadow** (yedek) mekanizması işin sürekliliğini güvenceye alırken, **Interface Fabric** (API sözleşmeleri, gözlemlenebilirlik, güvenlik, veri ve dağıtım standartları) heterojen teknolojiler arasında bütünlüğü korur.

Matematiksel gösterimler, r parametresinin (örn. 0.95/0.90/0.80) *demokratikleşme hızı* ve *kontrol yoğunluğu* üzerinde belirleyici olduğunu; **sahiplik derinliği** k 'nın ise yapısal bir *sağlık metriği* olarak işlev gördüğünü göstermektedir. $k > 1$ gözlemi, teknik/örgütsel inceleme tetikleyicisi olarak ele alınmalıdır. Modelin **Mod-A (Kalıcı Paylaşım)** ve **Mod-B (Emanet Kapasite)** varyantları, açık kaynak/kurumsal ekosistemlerle startup/scale-up bağlamlarının farklı ihtiyaçlarını pratik parametrelerle ($R_{\min}$, $\varepsilon_{\min}$) eşleyebilmemizi sağlar.

Son tahlilde Fraktal Sahiplik, “her yerde mükemmel kod” vaadi değil; **doğru anda, doğru yerde, doğru büyüklükte** müdahaleyi mümkün kılan bir *organizasyonel düzenektir*. Kod kalitesi; bu düzenekle birlikte işletilen standartlar ve kalite süreçlerinin (Interface Fabric, test ve yönetim) sonucudur. Amaç olarak mükemmel kaliteyi değil ancak kabul edilebilir kaliteyi, müşteriye hızlı ve ucuz bir şekilde sunmayı amaçlar.